



CAPAS DE SOFTWARE Y BLOQUEO MUTUO

APARICIO PONCE ROBERTO MANUEL
VAZQUEZ BLANCAS CESAR SAID
DIAZ TORRES JONATHAN SAMUEL
CORONA HERNÁNDEZ MARTIN RAFAEL



Interbloqueos

Definición:

Es un escenario donde múltiples procesos intentan acceder a una cantidad limitada de recursos y terminan bloqueados, esperando a que otros recursos estén disponibles.

Ejemplo:

Es comparable a la situación de dos trenes que se detienen en un cruce, incapaces de avanzar hasta que uno de ellos despeje el camino.

Importancia:

Los interbloqueos dificultan que los procesos que operan de manera simultánea puedan finalizar sus tareas.

Objetivos del capítulo:

- Explicar qué son los interbloqueos.
- Introducir estrategias para prevenir y evitar que ocurran.

Modelo del sistema

Recursos del Sistema: Los recursos en un sistema informático son variados e incluyen espacio de memoria, ciclos de CPU, dispositivos de entrada/salida (E/S) como impresoras y unidades de DVD. Cada tipo de recurso puede tener múltiples instancias disponibles. Por ejemplo, en un sistema con dos impresoras, ambas impresoras pertenecen a la misma clase de recurso, pero pueden considerarse como instancias separadas.

Asignación de recursos:

- Solicitud: Un proceso pide una instancia de un recurso. Si el recurso no está disponible en ese momento, el proceso debe esperar hasta que quede libre.
- Uso: Cuando el recurso es asignado, el proceso lo utiliza (por ejemplo, en el caso de una impresora, el proceso puede imprimir).
- Liberación: Al finalizar su tarea, el proceso libera el recurso, permitiendo que otros procesos puedan utilizarlo.

Llamadas al Sistema: Las solicitudes y liberaciones de recursos se gestionan mediante llamadas al sistema. Ejemplos incluyen: `request()`, `release()`: Para solicitar y liberar dispositivos. `open()`, `close()`: Para abrir y cerrar archivos. `allocate()`, `free()`: Para asignar y liberar memoria.

Tablas del Sistema: El sistema mantiene tablas para registrar el estado de cada recurso (libre o asignado) y para llevar un seguimiento de los procesos que tienen asignados recursos.

Caracterización de los interbloqueos

Definición:

Un interbloqueo ocurre cuando un grupo de procesos queda bloqueado, ya que cada uno espera un recurso que está siendo utilizado por otro proceso dentro del mismo grupo. Esto genera un ciclo de dependencias circulares, impidiendo que alguno de los procesos pueda continuar.

Consecuencias:

Un interbloqueo provoca que los procesos afectados nunca finalicen su ejecución, lo que resulta en la inmovilización de dichos procesos y en una utilización ineficiente de los recursos del sistema. Esto puede reducir el rendimiento del sistema y, en situaciones extremas, llevar a su completa paralización.

Condiciones necesarias para un interbloqueo

Exclusión mutua:

Debe haber al menos un recurso que no sea compartido, lo que significa que únicamente un proceso puede utilizarlo en un momento dado.

Retención y espera:

Un proceso debe mantener al menos un recurso mientras espera obtener otros recursos adicionales que están siendo utilizados por otros procesos.

Sin desalojo:

Los recursos no pueden ser retirados de forma forzada a los procesos que los están utilizando. Solo el proceso que retiene un recurso puede liberarlo voluntariamente después de completar su tarea.

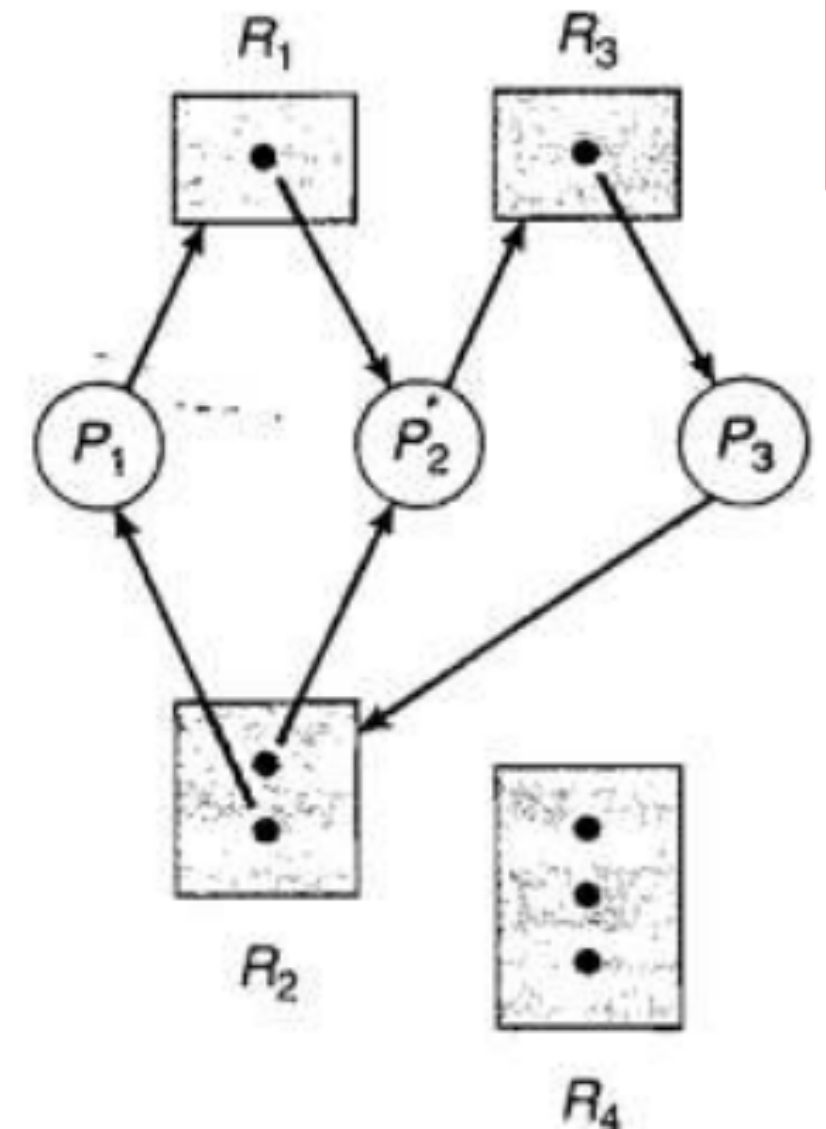
Espera circular:

Debe haber un conjunto de procesos $\{P_0, P_1, \dots, P_n\}$ donde cada proceso P_i espera un recurso que está siendo utilizado por el siguiente proceso P_{i+1} , y el último proceso P_n espera un recurso retenido por el primer proceso P_0 . Esto genera un ciclo de dependencia circular.

Grafo de asignacion de recursos

Definición:

Un grafo de asignación de recursos es una representación visual que ilustra las relaciones entre los procesos y los recursos en un sistema. Este grafo facilita la visualización de cómo los recursos están asignados a los procesos y cuáles son los recursos solicitados por cada proceso.



Elementos del grafo:

- Vértices: Los vértices representan tanto los procesos (P_i) como los recursos (R_j).
- Aristas de solicitud: Una arista dirigida desde un proceso P_i hacia un recurso R_j indica que el proceso P_i ha solicitado el recurso R_j y está esperando por él.
- Aristas de asignación: Una arista dirigida desde un recurso R_j hacia un proceso P_i muestra que el recurso R_j ha sido asignado al proceso P_i .

Ejemplos:

- Grafo sin ciclo: Si el grafo no tiene ciclos, significa que ningún proceso está en estado de interbloqueo.
- Grafo con ciclo: Si el grafo contiene un ciclo, podría haber un interbloqueo. Cada proceso en el ciclo está esperando un recurso que otro proceso en el ciclo retiene.

Interpretación:

La presencia de un ciclo en el grafo de asignación de recursos es una condición necesaria y suficiente para que exista un interbloqueo. Si hay un ciclo, hay interbloqueo; si no hay ciclo, no hay interbloqueo.

.

Metodos para tratar los interbloqueos

Prevenir interbloqueos:

Es posible utilizar un protocolo para garantizar que el sistema nunca entre en un estado de interbloqueo. Esto se logra evitando que se cumpla una o más de las condiciones necesarias para que ocurra un interbloqueo.

Ejemplos:

- Política de asignación ordenada: Los procesos deben solicitar los recursos en un orden preestablecido, lo que previene la espera circular.
- Restricciones en solicitudes: Se imponen límites a las solicitudes de recursos para que los procesos no puedan retener recursos mientras esperan otros.

Detectar y recuperar:

Permitir que el sistema entre en un estado de interbloqueo, pero implementar mecanismos para detectarlo y recuperarse de él.

- Detección: Se utilizan algoritmos que analizan el estado del sistema para identificar ciclos en el grafo de asignación de recursos.
- Recuperación:
 - Abortar procesos: Finalizar uno o más procesos para romper el ciclo de interbloqueo.
 - Liberar recursos: Forzar la liberación de recursos que están siendo retenidos por los procesos.

Ignorar el problema:

Actuar como si los interbloques nunca ocurrieran. Este enfoque es común en muchos sistemas operativos, como Unix y Windows, donde la responsabilidad de evitar los interbloques recae en los desarrolladores de aplicaciones.

Ejemplo:

- Reiniciar manualmente: En situaciones extremas, los administradores del sistema pueden tener que reiniciar manualmente los procesos o el sistema para resolver un interbloqueo.

Prevención de interbloqueos:

Son métodos y algoritmos específicos creados para asegurar que al menos una de las condiciones necesarias para el interbloqueo no se cumpla.

Ejemplos:

- Protocolos de adquisición y liberación: Diseñar el sistema de forma que los procesos soliciten y liberen recursos de manera que no puedan formar ciclos.

Evasión de interbloqueos:

Este enfoque requiere información adicional sobre los recursos que los procesos necesitarán y la duración de los mismos.

Algoritmo del banco:

Garantiza que los recursos se asignen de manera que se pueda satisfacer una solicitud sin que el sistema entre en un estado de interbloqueo.

Detección y recuperación:

- Algoritmos de detección:
- Estos algoritmos revisan periódicamente el estado del sistema para identificar ciclos en el grafo de asignación de recursos.
- Recuperación:
- Son estrategias diseñadas para resolver los interbloqueos una vez detectados, tales como abortar procesos o liberar recursos de manera forzada.

Prevención de interbloqueos

Interbloqueo (Deadlock):

Es una condición en la que dos o más procesos no pueden continuar porque cada uno espera que otro libere un recurso necesario.

Exclusión mutua:

Se refiere a la regla de que solo un proceso puede acceder a un recurso crítico al mismo tiempo, evitando interferencias entre procesos.

Retención y espera:

Es una situación en la que un proceso que ya posee al menos un recurso permanece en espera de recursos adicionales que están siendo utilizados por otros procesos.



Para prevenir interbloqueos, es necesario evitar al menos una de las siguientes condiciones:

1. Exclusión mutua: Aunque es difícil eliminarla por completo, se puede reducir permitiendo el acceso compartido a los recursos cuando sea posible.
 2. Retención y espera: Requerir que los procesos soliciten todos los recursos necesarios al mismo tiempo, en lugar de hacerlo gradualmente.
 3. No desalojo: Permitir que un proceso libere recursos si no puede obtener todos los que necesita.
 4. Espera circular: Establecer un orden lineal para los recursos y obligar a los procesos a solicitarlos siguiendo ese orden.
- 

Sin desalojo (No Preemption):
Implica que un recurso asignado a un proceso no puede ser retirado de forma forzada hasta que el proceso termine su tarea con él. Para prevenir interbloqueos, se puede implementar la opción de que los procesos liberen los recursos asignados si no logran obtener todos los recursos requeridos.

Espera circular (Circular Wait):
Ocurre cuando un grupo de procesos $\{P1, P2, \dots, Pn\}$ forma un ciclo en el que:

- P1 espera un recurso que tiene P2,
- P2 espera un recurso que tiene P3,
- y así sucesivamente hasta que Pn espera un recurso que posee P1.

Para prevenir esta condición, se puede establecer un orden global en los recursos y exigir que los procesos los soliciten siguiendo un orden ascendente.

Evasión de Interbloqueos

Consiste en tomar decisiones anticipadas sobre las solicitudes de recursos para garantizar que el sistema no alcance un estado de interbloqueo. Esto se logra empleando algoritmos, como el del banquero, que analizan si conceder una solicitud podría llevar a una situación insegura.

Estado Seguro

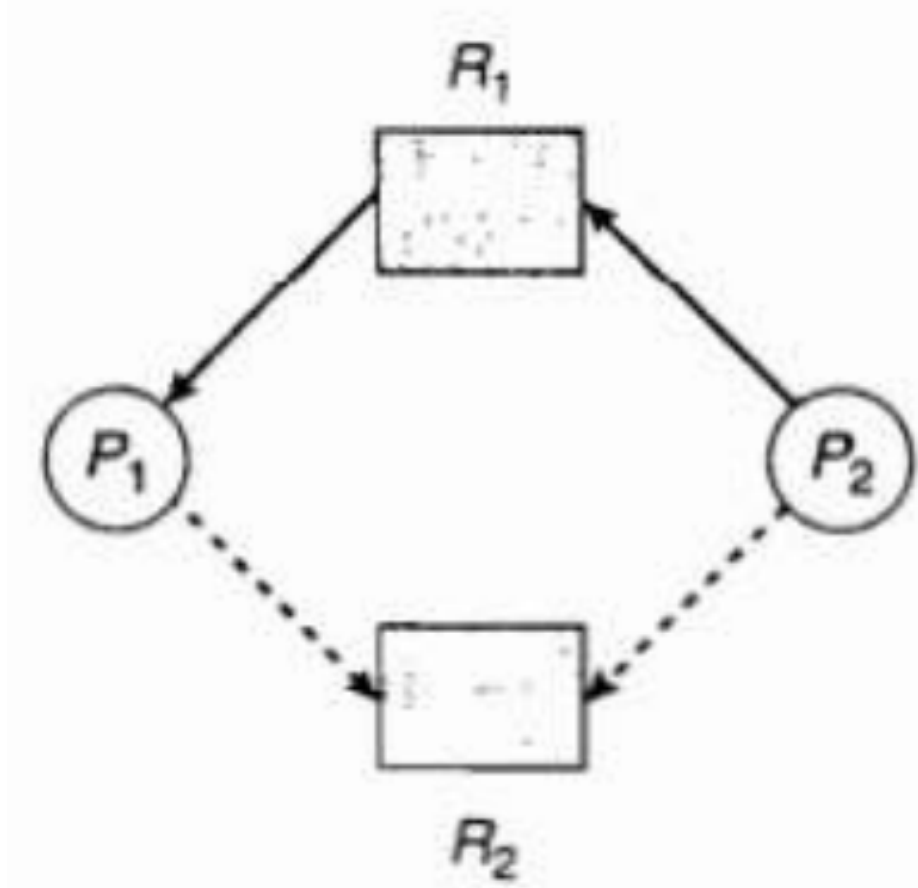
Es una condición en la que el sistema puede distribuir los recursos entre los procesos de manera que todos logren completar su ejecución sin generar interbloqueos. En este estado, existe una secuencia definida de procesos que garantiza que cada uno terminará su tarea en algún momento.

Grafo de asignación de recursos

Es una representación gráfica que muestra cómo se asignan y solicitan los recursos en un sistema. Los nodos representan procesos y recursos, mientras que los arcos dirigidos indican las relaciones entre ellos:

- Arco de Solicitud: Un proceso solicita un recurso.
- Arco de Asignación: Un recurso ha sido asignado a un proceso.

La presencia de un ciclo en este grafo es un indicador potencial de un interbloqueo.



Algoritmo del banquero

Propuesto por Edsger Dijkstra, este algoritmo es una técnica para evitar interbloqueos. Su funcionamiento implica simular la asignación de recursos antes de realizarla para determinar si la nueva asignación puede conducir a un estado seguro o inseguro.

Pasos principales:

1. Simulación: Antes de asignar recursos, el algoritmo verifica si la solicitud puede ser satisfecha sin dejar el sistema en un estado inseguro.
2. Estado Seguro: Si tras la asignación existe al menos una secuencia en la que todos los procesos puedan completarse, el sistema permanece en estado seguro.
3. Denegación de Solicitud: Si la asignación podría llevar a un estado inseguro, la solicitud de recursos es denegada.

Este método es especialmente útil en sistemas con una cantidad fija de recursos y donde se pueden anticipar las necesidades máximas de los procesos.

Funcionamiento del algoritmo del banquero

1. Solicitud de Recursos:

2. Un proceso hace una solicitud para un conjunto específico de recursos.

3. Simulación:

4. El sistema simula la asignación de los recursos solicitados al proceso, ajustando temporalmente la disponibilidad de recursos y el estado actual.

5. Evaluación del Estado:

- Se evalúa si el sistema permanece en un estado seguro.
- Esto implica verificar si existe una secuencia segura en la que todos los procesos puedan finalizar sin causar interbloqueos.

6. Decisión:

- Estado Seguro: Si el sistema sigue siendo seguro tras la simulación, los recursos son asignados al proceso.
- Estado Inseguro: Si el sistema podría entrar en un estado inseguro, la solicitud de recursos es denegada, y el proceso debe esperar.

Este enfoque garantiza que los recursos solo se asignen de manera que el sistema evite interbloqueos, manteniendo siempre una configuración estable y controlada.

Algoritmo de Seguridad

Inicialización: Se generan dos vectores, Work (que es una copia de Available) y Finish (que se inicializa en falso).

Búsqueda de Proceso: Se busca un proceso i que aún no se haya completado ($\text{Finish}[i] == \text{false}$) y cuya cantidad de recursos necesarios ($\text{Need}[i]$) pueda ser cubierta con los recursos disponibles (Work).

Asignación de Recursos: Si se encuentra un proceso que cumpla con estas condiciones, se asignan los recursos a dicho proceso, actualizando Work, y se marca como completado al proceso ($\text{Finish}[i] = \text{true}$).

Este paso se repite.

Verificación de Seguridad: Si todos los procesos tienen el valor $\text{Finish}[i]$ igual a true, el sistema está en un estado seguro.

Algoritmo de solicitud de recursos

Verificación de Necesidades: Se comprueba que la solicitud de recursos del proceso P_i no supere sus necesidades máximas ($Request[i] \leq Need[i]$). Si es mayor, se genera un error.

Disponibilidad de Recursos: Se verifica si los recursos solicitados están disponibles ($Request[i] \leq Available$). Si no es así, el proceso debe esperar.

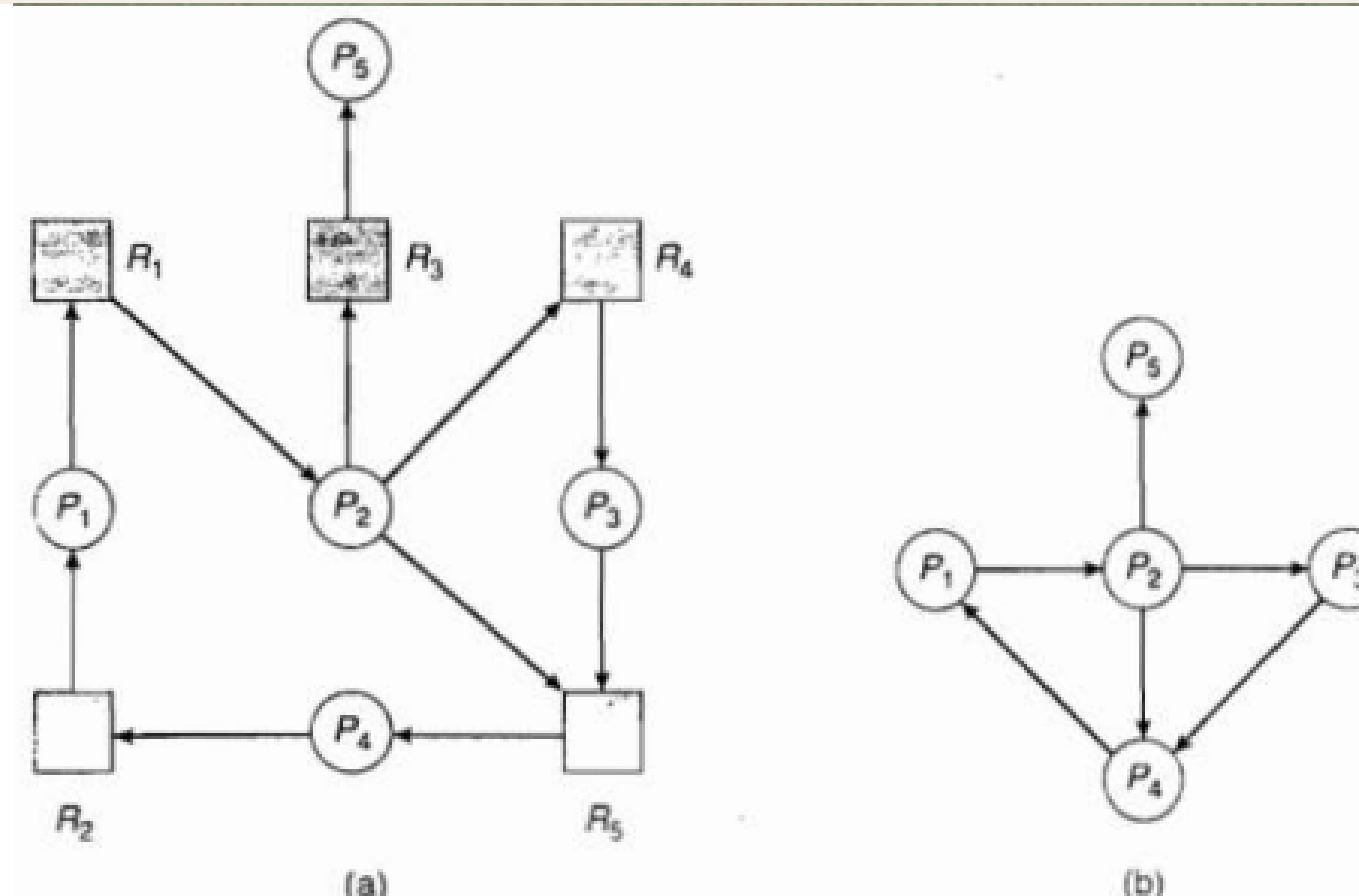
Asignación Simulada: Si los recursos están disponibles, se asignan temporalmente al proceso y se actualizan las matrices *Available*, *Allocation* y *Need*.

Verificación de Seguridad: Se ejecuta el algoritmo de seguridad para verificar si el nuevo estado sigue siendo seguro. Si es seguro, la asignación se completa; de lo contrario, el proceso debe esperar.

Detección de interbloqueos

Detección de Interbloqueos con una Sola Instancia de Cada Tipo de Recurso:

Este método emplea un grafo de espera en el cual una arista de P_i a P_j indica que el proceso P_i está esperando un recurso que posee el proceso P_j . Un interbloqueo ocurre únicamente si el grafo de espera contiene un ciclo, ya que esto implica que los procesos están esperando de forma circular por los recursos, sin poder avanzar.



Para sistemas con múltiples instancias de cada tipo de recurso, se utiliza una versión extendida del algoritmo de detección de interbloqueos, similar al algoritmo del banquero:

Inicialización: Se crean los vectores Work (copia del vector Available) y Finish (todos inicializados como false).

1. Búsqueda de Proceso: Se busca un proceso i que pueda ser completado, es decir, cuyo requerimiento de recursos ($Request[i]$) sea menor o igual a los recursos disponibles en Work.
2. Asignación de Recursos: Si se encuentra tal proceso, los recursos se asignan provisionalmente, actualizando el vector Work y marcando $Finish[i]$ como true. Luego, se repite el paso 1.
3. Determinación de Interbloqueo: Si al final de este proceso, cualquier valor de $Finish[i]$ sigue siendo false, significa que el sistema está en un estado de interbloqueo, ya que no todos los procesos pueden completarse.

Utilizacion

Frecuencia de Detección:

La frecuencia con la que se detectan los interbloqueos depende de cuán a menudo ocurren y de cuántos procesos se ven involucrados. Si los interbloqueos son comunes, el algoritmo de detección debería ejecutarse con mayor frecuencia para identificar y manejar los ciclos de interbloqueo de manera eficiente.

Recuperación de un Interbloqueo:

Existen diferentes enfoques para recuperar el sistema después de detectar un interbloqueo:

1. Terminación de Procesos:

- Interrumpir todos los procesos involucrados en el interbloqueo.
- Interrumpir los procesos uno a uno hasta romper el ciclo de interbloqueo, eliminando gradualmente los procesos que forman parte del ciclo.

2. Apropiación de Recursos:

- Selección de una víctima: Determinar qué recursos deben ser apropiados y de qué procesos, para romper el ciclo de interbloqueo.
- Anulación: Interrumpir el proceso seleccionado y reiniciarlo después de liberar los recursos que tenía asignados.

3. Inanición:

4. Es importante asegurarse de que un proceso no sea seleccionado continuamente como víctima de la apropiación de recursos, evitando la inanición, es decir, que el proceso nunca pueda completarse debido a la constante interrupción.

Apropiación de recursos

Apropiación de Recursos:

Este método implica liberar los recursos de los procesos seleccionados para resolver un interbloqueo. La liberación de estos recursos permite que otros procesos continúen su ejecución y, al romper el ciclo de espera, se restablece el sistema a un estado funcional.

Resumen:

Un interbloqueo ocurre cuando dos o más procesos están esperando indefinidamente recursos que están siendo retenidos por otros procesos, creando un ciclo de espera en el que ninguno de los procesos puede avanzar.

Métodos para tratar interbloqueos:

1. Prevención o Evasión:

- Diseñar el sistema para evitar que las condiciones necesarias para un interbloqueo se presenten, o bien para evitar que el sistema entre en un estado peligroso.

2. Detección y Recuperación:

- Detectar los interbloqueos cuando ocurren y aplicar estrategias para recuperarse de ellos, como la terminación de procesos o la apropiación de recursos.

3. Ignorar el Problema y Reiniciar:

- En algunos sistemas, los interbloqueos pueden ser ignorados, y se opta por reiniciar el sistema o los procesos involucrados cuando un interbloqueo es detectado.

Modelacion de bloqueos

Introducción al Interbloqueo:

El modelo de sistema de interbloqueo se basa en cómo los procesos solicitan, utilizan y liberan recursos. Los interbloqueos ocurren cuando los procesos necesitan acceder a los mismos recursos de entrada y salida de manera simultánea, creando una situación donde ninguno de los procesos puede avanzar.

Condiciones de Coffman para el Interbloqueo:

Estas condiciones son necesarias para que un interbloqueo se produzca en un sistema de múltiples procesos y recursos:

1. Exclusión mutua: Solo un proceso puede usar un recurso en un momento dado.
2. Retención y espera: Los procesos mantienen recursos que ya han sido asignados mientras esperan que otros recursos estén disponibles.
3. No apropiación: Los recursos no pueden ser forzadamente retirados de un proceso, incluso si otro proceso los necesita.
4. Espera circular: Existe una cadena de procesos, cada uno esperando un recurso que está siendo retenido por otro proceso en la cadena, creando un ciclo de espera.

Representación Gráfica de Interbloqueos:

En la representación gráfica de interbloqueos, los procesos se representan mediante círculos, y los recursos mediante cuadrados. Las flechas que van desde los recursos hacia los procesos indican que el proceso está utilizando ese recurso, mientras que las flechas que van desde los procesos hacia los recursos indican que el proceso está solicitando dicho recurso.

Conclusión:

Un interbloqueo solo ocurre si se cumplen las cuatro condiciones de Coffman:

1. Exclusión mutua: Solo un proceso puede usar un recurso a la vez.
2. Retención y espera: Un proceso mantiene un recurso mientras espera por otros.
3. No apropiación: Los recursos no pueden ser retirados de un proceso de manera forzada.
4. Espera circular: Existe un ciclo en el que los procesos están esperando recursos retenidos por otros procesos.

Si alguna de estas condiciones no se cumple, el sistema no entra en un estado de interbloqueo.

Conclusion

Los interbloqueos son cruciales para la eficiencia y seguridad de los sistemas operativos, ya que evitan la paralización total de los procesos. Estos se caracterizan por la espera circular de recursos y se producen cuando se presentan simultáneamente las cuatro condiciones específicas de Coffman: exclusión mutua, retención y espera, no apropiación y espera circular.

La prevención y el manejo adecuado de los interbloqueos son fundamentales para garantizar la estabilidad del sistema. Los métodos de prevención, evasión y detección son esenciales para abordar los interbloqueos, asegurando que el sistema siga funcionando correctamente.

Aunque los interbloqueos representan un desafío considerable, con una gestión adecuada, se pueden aprovechar para optimizar y asegurar la continuidad operativa del sistema, evitando que afecten la ejecución de los procesos.

GRACIAS POR SU ATENCION